CSS — ZERO TO HERO

Complete Teaching Notes for Students

Properties • Values • Examples • Rules • Viva • Quick Revision

~ Yug Sardhara

1. What is CSS?

1. Definition: CSS means Cascading Style Sheets. It is used to control the appearance and layout of HTML.

2. Purpose / Why we use it: To separate presentation from content and create consistent, maintainable designs.

3. Beginner-Friendly Explanation: HTML creates the structure; CSS decides colors, spacing, fonts, sizes and layout.

4. Syntax:

```
selector { property: value; }
```

5. Example:

```
h1 { color: blue; font-size: 32px; }
```

6. Expected Output: The heading becomes blue and 32px.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
color	CSS	Text color.
font-size	CSS	Text size.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Use braces around declarations.

• Use : between property and value.

10. Common Mistakes:

• Using = instead of :.

• Forgetting ; between declarations.

11. Viva / Remember Points: CSS = styling + layout.

2. Why CSS?

1. Definition: CSS styles and arranges HTML content.

2. Purpose / Why we use it: For reuse, consistency, maintainability, responsive layouts and attractive design.

3. Beginner-Friendly Explanation: One CSS rule can style many matching elements instead of repeating formatting in HTML.

4. Syntax:

```
p { color: navy; font-size: 18px; }
```

5. Example:

```
p { color: navy; font-size: 18px; }
```

6. Expected Output: All paragraphs become navy and 18px.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
color	CSS	Text color.
font-size	CSS	Text size.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Prefer reusable rules.

• Use classes for reusable component styles.

10. Common Mistakes:

• Repeating the same style inline everywhere.

11. Viva / Remember Points: CSS saves repetition and keeps design consistent.

3. Types of CSS

1. Definition: CSS can be applied as Inline, Internal or External CSS.

2. Purpose / Why we use it: To choose where styles are written.

3. Beginner-Friendly Explanation: Inline is inside style=""; Internal is inside <style>; External is a separate .css file.

4. Syntax:

```
Inline: style="color:red;"
Internal: <style>p{color:red;}</style>
External: <link rel="stylesheet" href="style.css">
```

5. Example:

```
Inline: style="color:red;"
Internal: <style>p{color:red;}</style>
External: <link rel="stylesheet" href="style.css">
```

6. Expected Output: All three can style HTML; external CSS is most reusable for multiple pages.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
style	HTML attribute	Inline CSS.
rel	<link>	Relationship, e.g. stylesheet.
href	<link>	CSS file path.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• External CSS is normally preferred for larger projects.

10. Common Mistakes:

• Confusing style attribute with <style> element.

11. Viva / Remember Points: Inline = element; Internal = page; External = reusable file.

4. CSS Syntax

1. Definition: The structure used to write a CSS rule.

2. Purpose / Why we use it: To tell the browser which elements to style and what to change.

3. Beginner-Friendly Explanation: A selector chooses the target; declarations contain property and value.

4. Syntax:

```
selector { property: value; }
```

5. Example:

```
p { color: green; font-size: 18px; }
```

6. Expected Output: Paragraphs become green and 18px.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
selector	CSS	Chooses target.
property	CSS	Feature to change.
value	CSS	Setting.
;	CSS	Ends declaration.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Use : between property/value.

• Use ; between declarations.

10. Common Mistakes:

• color=red.

• Missing braces.

11. Viva / Remember Points: selector { property: value; }

5. Selector Concept

1. Definition: A selector identifies HTML element(s) that CSS should style.

2. Purpose / Why we use it: To target the correct elements.

3. Beginner-Friendly Explanation: Selectors are like addresses: p, .card and #title point to different targets.

4. Syntax:

```
p { color:blue; }
.card { border:1px solid; }
#title { font-size:30px; }
```

5. Example:

```
p { color:blue; }
.card { border:1px solid; }
#title { font-size:30px; }
```

6. Expected Output: Each matching target receives its style.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
p	Element selector	All p elements.
.card	Class selector	Elements with class card.
#title	ID selector	Element with id title.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Know element vs class vs ID.

10. Common Mistakes:

• Forgetting . or #.

11. Viva / Remember Points: Selector = who gets the CSS.

6. Element Selector

1. Definition: Selects elements by tag name.

2. Purpose / Why we use it: To style all elements of one HTML type.

3. Beginner-Friendly Explanation: p selects every paragraph; h1 selects every h1.

4. Syntax:

```
p { property: value; }
```

5. Example:

```
p { color: blue; }
```

6. Expected Output: Every paragraph becomes blue.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
color	CSS	Text color.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Do not use . or #.

10. Common Mistakes:

• Writing .p by mistake.

11. Viva / Remember Points: p = all paragraph tags.

7. Class Selector

1. Definition: Selects elements using class.

2. Purpose / Why we use it: To reuse the same style on many elements.

3. Beginner-Friendly Explanation: A class is reusable and begins with . in CSS.

4. Syntax:

```
.class-name { property: value; }
```

5. Example:

```
<p class="highlight">Important</p>
.highlight { background-color: yellow; }
```

6. Expected Output: The paragraph gets a yellow background.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
class	HTML	Class name.
background-color	CSS	Background color.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Many elements may share a class.

• Multiple classes are separated by spaces.

10. Common Mistakes:

• Forgetting the dot.

11. Viva / Remember Points: .className = reusable.

8. ID Selector

1. Definition: Selects an element using id.

2. Purpose / Why we use it: To target a specific normally-unique element.

3. Beginner-Friendly Explanation: An ID selector begins with #.

4. Syntax:

```
#id-name { property: value; }
```

5. Example:

```
<h1 id="title">Welcome</h1>
#title { color: darkblue; }
```

6. Expected Output: The heading becomes dark blue.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
id	HTML	Identifies an element.
color	CSS	Text color.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• An id should be unique in a document.

10. Common Mistakes:

• Using # incorrectly or duplicating ids.

11. Viva / Remember Points: #idName = specific target.

9. Universal Selector

1. Definition: Selects all elements.

2. Purpose / Why we use it: For base styles and resets.

3. Beginner-Friendly Explanation: The * selector means every element.

4. Syntax:

```
* { property: value; }
```

5. Example:

```
* { box-sizing: border-box; margin: 0; }
```

6. Expected Output: All elements receive the declarations.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
box-sizing	CSS	Controls sizing calculation.
margin	CSS	Outer spacing.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Use carefully because it affects everything.

10. Common Mistakes:

• Putting heavy styles on *.

11. Viva / Remember Points: * = all.

10. Grouping Selector

1. Definition: Styles multiple selectors with one rule.

2. Purpose / Why we use it: To avoid repeating identical declarations.

3. Beginner-Friendly Explanation: Separate selectors with commas.

4. Syntax:

```
h1, h2, p { color: navy; }
```

5. Example:

```
h1, h2, p { font-family: Arial, sans-serif; color: navy; }
```

6. Expected Output: All h1, h2 and p share the style.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
,	CSS syntax	Separates selectors.
font-family	CSS	Font family.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Use commas for grouping.

10. Common Mistakes:

• Using spaces instead of commas.

11. Viva / Remember Points: Many selectors, one rule.

11. Colors

1. Definition: CSS color properties control text, background and border colors.

2. Purpose / Why we use it: To create contrast, hierarchy and a consistent visual theme.

3. Beginner-Friendly Explanation: CSS supports color names, HEX, RGB/RGBA and HSL/HSLA.

4. Syntax:

```
color: value;
background-color: value;
border-color: value;
```

5. Example:

```
h1{color:#2563eb;} .card{background-color:rgb(245,245,245);border-color:black;}
```

6. Expected Output: Blue heading, light-gray background and black border.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
color	CSS	Text color.
background-color	CSS	Background color.
border-color	CSS	Border color.

8. Attribute → Belongs to → Meaning: These are CSS properties.

Common color formats

Property / Attribute	Belongs to	Meaning
Format	Belongs to	Meaning
Name	CSS value	red, blue, black etc.
HEX	CSS value	#2563eb
RGB	CSS value	rgb(37,99,235)
RGBA	CSS value	RGB + alpha/transparency
HSL	CSS value	Hue + saturation + lightness
HSLA	CSS value	HSL + alpha

9. Important Rules: Choose readable contrast; alpha can control transparency.

10. Common Mistakes: Confusing color with background-color.

11. Viva / Remember Points: color = text; background-color = background.

12. Background

1. Definition: Background properties control what appears behind an element.

2. Purpose / Why we use it: To add colors/images and control repeat, size and position.

3. Beginner-Friendly Explanation: background-color paints the background; background-image places an image behind content.

4. Syntax:

```
background-color:value;
background-image:url("image.jpg");
```

5. Example:

```
.hero{background-image:url("banner.jpg");background-repeat:no-repeat;background-
size:cover;background-position:center;}
```

6. Expected Output: A centered, non-repeating image fills the hero area.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
background-color	CSS	Background color.

background-image	CSS	Background image.
background-repeat	CSS	repeat/no-repeat/repeat-x/repeat-y.
background-size	CSS	auto/cover/contain or lengths.
background-position	CSS	center/top/right etc.

8. Attribute → Belongs to → Meaning: All listed items are background CSS properties.

9. Important Rules: cover fills the box but can crop; contain keeps the whole image but may leave space.

10. Common Mistakes: Wrong image path or forgetting url().

11. Viva / Remember Points: Background = area behind content.

13. Text & Fonts

1. Definition: Typography properties control font family, size, style, weight, alignment and spacing.

2. Purpose / Why we use it: To improve readability and visual hierarchy.

3. Beginner-Friendly Explanation: font-* controls typeface/size/weight; text-* controls alignment/decoration/spacing.

4. Syntax:

```
font-family:value;
font-size:value;
font-weight:value;
text-align:value;
```

5. Example:

```
p{font-family:Arial,sans-serif;font-size:18px;font-style:italic;font-weight:700;line-height:1.5;text-align:center;text-decoration:underline;letter-spacing:1px;word-spacing:4px;}
```

6. Expected Output: Centered, italic, bold, underlined 18px text with changed spacing.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
font-family	CSS	Font and fallbacks.
font-size	CSS	Text size.
font-style	CSS	normal/italic/oblique.
font-weight	CSS	Thickness: normal, bold or 100–900.
line-height	CSS	Line spacing.
text-align	CSS	left/center/right/justify.
text-decoration	CSS	underline/none/line-through.
letter-spacing	CSS	Character spacing.
word-spacing	CSS	Word spacing.
text-transform	CSS	uppercase/lowercase/capitalize.

8. Attribute → Belongs to → Meaning: These are CSS typography properties.

font-size values

Property / Attribute	Belongs to	Meaning
Category	Belongs to	Meaning
Absolute keyword	font-size	xx-small, x-small, small, medium,

		large, x-large, xx-large, xxx-large.
Relative keyword	font-size	smaller, larger.
Length	font-size	px, rem, em and other supported lengths.
Percentage	font-size	Relative percentage such as 120%.

9. Important Rules: Use font fallbacks such as Arial, sans-serif. 1rem is commonly based on the root font size.

10. Common Mistakes: Confusing font-weight and font-size.

11. Viva / Remember Points: family = typeface; size = size; weight = thickness; align = position of text.

14. Width & Height

1. Definition: width and height control an element's dimensions.

2. Purpose / Why we use it: To size boxes and components.

3. Beginner-Friendly Explanation: Units include px, %, rem, em, vw, vh and auto; min/max can limit dimensions.

4. Syntax:

```
width:value;
height:value;
```

5. Example:

```
.box{width:80%;height:300px;}
```

6. Expected Output: The box uses 80% of its containing block width and 300px height.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
width	CSS	Preferred width.
height	CSS	Preferred height.
min-width	CSS	Minimum width.
max-width	CSS	Maximum width.
min-height	CSS	Minimum height.
max-height	CSS	Maximum height.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• max-width is useful for responsive content.

• Fixed heights can cause overflow.

10. Common Mistakes:

• Assuming 100% means viewport width.

• Using fixed height for growing content.

11. Viva / Remember Points: width/height = dimensions; min/max = limits.

15. Border

1. Definition: Border draws a line around an element.

2. Purpose / Why we use it: To frame, separate or emphasize content.

3. Beginner-Friendly Explanation: A visible border normally uses width, style and color.

4. Syntax:

```
border:width style color;
```

5. Example:

```
.card{border:2px solid #333;border-radius:10px;}
```

6. Expected Output: A dark border with rounded corners appears.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
border	CSS	Width + style + color shorthand.
border-width	CSS	Thickness.
border-style	CSS	solid/dashed/dotted/double/none.
border-color	CSS	Color.
border-radius	CSS	Rounded corners.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• A visible border needs an appropriate border-style.

10. Common Mistakes:

• border:2px red without style.

11. Viva / Remember Points: Border = width + style + color.

16. Margin

1. Definition: Margin is space outside an element's border.

2. Purpose / Why we use it: To separate neighboring elements.

3. Beginner-Friendly Explanation: Think of it as outside space around the box.

4. Syntax:

```
margin:top right bottom left;
```

5. Example:

```
h2{margin:20px 0;}
```

6. Expected Output: 20px top/bottom and 0 left/right.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
margin	CSS	Outer spacing shorthand.
margin-top	CSS	Top space.
margin-right	CSS	Right space.
margin-bottom	CSS	Bottom space.
margin-left	CSS	Left space.
auto	CSS value	Can distribute horizontal free space in suitable layouts.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• 1 value=all; 2=vertical/horizontal; 3=top/horizontal/bottom; 4=top/right/bottom/left.

10. Common Mistakes:

• Using margin for inner space.

11. Viva / Remember Points: Margin = outside.

17. Padding

1. Definition: Padding is space between content and border.

2. Purpose / Why we use it: To create inner breathing room.

3. Beginner-Friendly Explanation: Think of it as inside space.

4. Syntax:

```
padding:top right bottom left;
```

5. Example:

```
button{padding:10px 20px;}
```

6. Expected Output: 10px top/bottom and 20px left/right.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
padding	CSS	Inner spacing shorthand.
padding-top	CSS	Top inner space.
padding-right	CSS	Right inner space.
padding-bottom	CSS	Bottom inner space.
padding-left	CSS	Left inner space.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Padding is inside the border and cannot be negative.

10. Common Mistakes:

• Using margin when space is needed inside.

11. Viva / Remember Points: Padding = inside.

18. Box Model

1. Definition: Every element is a box: Content → Padding → Border → Margin.

2. Purpose / Why we use it: It explains size and spacing calculations.

3. Beginner-Friendly Explanation: Imagine a gift: content is the item, padding is inner space, border is the wall and margin is outside space.

4. Syntax:

```
box-sizing:content-box;
box-sizing:border-box;
```

5. Example:

```
.box{width:200px;padding:20px;border:5px solid;margin:10px;box-sizing:border-box;}
```

6. Expected Output: With border-box, the declared 200px width includes content, padding and border.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
width	CSS	Declared width.
padding	CSS	Inner space.
border	CSS	Boundary.
margin	CSS	Outer space.
box-sizing	CSS	Controls sizing calculation.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• content-box adds padding/border outside declared content size; border-box includes them.

10. Common Mistakes:

• Misunderstanding total width.

11. Viva / Remember Points: Content → Padding → Border → Margin.

19. Display

1. Definition: display controls how an element participates in layout.

2. Purpose / Why we use it: To use block, inline, inline-block, flex, grid, none and other layout modes.

3. Beginner-Friendly Explanation: display tells the browser how the element's box should behave.

4. Syntax:

```
display:value;
```

5. Example:

```
p{display:block;} span{display:inline-block;} .container{display:flex;}
```

6. Expected Output: p behaves as block, span as inline-block and container becomes a flex container.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
display:block	CSS	Block behavior.
display:inline	CSS	Inline flow.
display:inline-block	CSS	Inline flow with box dimensions.
display:none	CSS	Not rendered and no layout space.
display:flex	CSS	Flex container.
display:grid	CSS	Grid container.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Direct children of a flex container become flex items.

10. Common Mistakes:

• Expecting display:none to leave space.

11. Viva / Remember Points: display:flex starts Flexbox.

20. Float

1. Definition: float moves an element left or right and lets inline content wrap around it.
2. Purpose / Why we use it: Historically used for layout; still useful for image/text wrapping.
3. Beginner-Friendly Explanation: Float is not the modern replacement for Flexbox.
4. Syntax:

```
float:left;
float:right;
float:none;
```

5. Example:

```
img{float:left;margin-right:15px;}
```

6. Expected Output: Text can wrap around the image.
7. Attributes Used:

Property / Attribute	Belongs to	Meaning
float	CSS	left/right/none and logical values where supported.
margin-right	CSS	Space beside image.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.
9. Important Rules:

• Floats affect normal-flow behavior.

• Use clear when necessary.

10. Common Mistakes:

• Using float for every modern layout.

11. Viva / Remember Points: float = move + wrap.

21. Clear

1. Definition: clear controls whether an element can sit beside floated elements.
2. Purpose / Why we use it: To place content below floats.
3. Beginner-Friendly Explanation: clear:both is common when both sides are floated.
4. Syntax:

```
clear:left;
clear:right;
clear:both;
```

5. Example:

```
img{float:left;} footer{clear:both;}
```

6. Expected Output: Footer appears below the float.
7. Attributes Used:

Property / Attribute	Belongs to	Meaning
clear	CSS	Clears left/right/both floats.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• clear affects the cleared element, not the float.

10. Common Mistakes:

• Thinking clear removes the float.

11. Viva / Remember Points: float moves; clear stops wrapping.

22. Position

1. Definition: position controls how an element is positioned.

2. Purpose / Why we use it: For overlays, badges, sticky headers, fixed controls and precise placement.

3. Beginner-Friendly Explanation: static is normal; relative keeps space; absolute leaves normal flow; fixed is viewport-oriented; sticky changes behavior during scroll.

4. Syntax:

```
position:value;
top:value;
right:value;
bottom:value;
left:value;
```

5. Example:

```
.header{position:sticky;top:0;} .card{position:relative;}
.badge{position:absolute;top:10px;right:10px;}
```

6. Expected Output: Header can stick at top; badge sits near card's top-right.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
position	CSS	static/relative/absolute/fixed/sticky.
top/right/bottom/left	CSS	Offsets positioned elements.
z-index	CSS	Stacking order when applicable.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• A common pattern is parent relative + child absolute.

10. Common Mistakes:

• Using absolute without understanding its containing block.

11. Viva / Remember Points: relative keeps space; absolute leaves normal flow.

23. Images

1. Definition: CSS controls image dimensions, fitting, shape, opacity and effects.

2. Purpose / Why we use it: To make images responsive and suitable for layouts.

3. Beginner-Friendly Explanation: HTML <img> supplies the image; CSS controls presentation.

4. Syntax:

```
img{width:100%;height:auto;}
```

5. Example:

```
img{width:100%;max-width:400px;height:250px;object-fit:cover;border-radius:12px;}
```

6. Expected Output: Image can shrink, is limited to 400px, covers its box and has rounded corners.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
width	CSS	Image width.
max-width	CSS	Maximum width.
height	CSS	Image height.
object-fit	CSS	How image fits its box.
object-position	CSS	Image content position.
border-radius	CSS	Rounded corners.
opacity	CSS	Transparency.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• cover may crop; contain keeps whole image.

10. Common Mistakes:

• Stretching images with mismatched dimensions.

11. Viva / Remember Points: width:100%; height:auto is a common responsive starting point.

24. Lists

1. Definition: CSS list properties control markers and their placement.

2. Purpose / Why we use it: To customize ordered and unordered lists.

3. Beginner-Friendly Explanation: HTML creates lists; CSS controls marker appearance.

4. Syntax:

```
ul{list-style-type:square;}
ol{list-style-type:upper-roman;}
```

5. Example:

```
ul{list-style-type:square;list-style-position:inside;} ol{list-style-type:upper-roman;}
```

6. Expected Output: ul has square bullets; ol uses Roman numerals.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
list-style-type	CSS	Marker type.
list-style-position	CSS	inside/outside.
list-style-image	CSS	Image marker.
list-style	CSS	Shorthand.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• list-style-type changes markers, not layout.

10. Common Mistakes:

• Expecting it to add spacing.

11. Viva / Remember Points: ul → bullets; ol → numbering.

25. Links

1. Definition: CSS styles anchor elements and their states.

2. Purpose / Why we use it: To control link color, decoration and interaction.

3. Beginner-Friendly Explanation: Links can have different styles for normal, visited, hover and active states.

4. Syntax:

```
a:state{property:value;}
```

5. Example:

```
a{color:blue;text-decoration:none;} a:hover{text-decoration:underline;} a:visited{color:purple;}
```

6. Expected Output: Normal links are blue; hover underlines; visited can change color.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
a:link	Pseudo-class	Unvisited.
a:visited	Pseudo-class	Visited.
a:hover	Pseudo-class	Pointer over.
a:active	Pseudo-class	Being activated.

8. Attribute → Belongs to → Meaning: The table above lists the properties/attributes used in the example.

9. Important Rules:

• Remember LVHA order.

10. Common Mistakes:

• Styling only hover.

11. Viva / Remember Points: link → visited → hover → active.

26. Flexbox

1. Definition: Flexbox is a one-dimensional layout system for arranging items in a row or column.

2. Purpose / Why we use it: For navigation bars, left-right layouts, centering, cards and flexible spacing.

3. Beginner-Friendly Explanation: Put display:flex on the parent. Direct children become flex items. Then control direction, alignment, spacing and child sizing.

4. Syntax:

```
container{display:flex;flex-direction:row;justify-content:space-between;align-items:center;}
```

5. Example:

```
<main class="layout">
<section class="left">Left</section>
<section class="right">Right</section>
</main>
.layout{display:flex;gap:20px;}
.left{flex:2;}
.right{flex:3;}
```

6. Expected Output: Two sections appear side by side; flexible space is distributed roughly in a 2:3 ratio.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
display:flex	Parent	Creates flex container.
flex-direction	Parent	row/row-reverse/column/column-reverse.
justify-content	Parent	Main-axis distribution.
align-items	Parent	Cross-axis alignment.
gap	Parent	Space between items.
flex-wrap	Parent	Allows wrapping.
flex	Child	Flexible grow/shrink/basis.
align-self	Child	Individual cross-axis alignment.
order	Child	Visual order.

8. Attribute → Belongs to → Meaning: Parent properties control the container; child properties control individual items.

9. Important Rules: With row, main axis is horizontal; with column, main axis is vertical. justify-content works on the main axis; align-items on the cross axis.

10. Common Mistakes: Confusing justify-content and align-items; forgetting axes change with flex-direction.

11. Viva / Remember Points: display:flex starts Flexbox; justify-content = main axis; align-items = cross axis; gap = spacing; flex = child sizing.

Flexbox Directions

Property / Attribute	Belongs to	Meaning
row	CSS value	Left → right in normal writing direction.
row-reverse	CSS value	Right → left.
column	CSS value	Top → bottom.
column-reverse	CSS value	Bottom → top.

27. Complete Layout Exercise

1. Definition: A complete layout combines selectors, colors, typography, box model, display and Flexbox.

2. Purpose / Why we use it: To connect individual CSS concepts into a practical webpage.

3. Beginner-Friendly Explanation: Build in layers: base styles → header → nav → main Flexbox → cards → footer.

4. Syntax: HTML structure + CSS rules.

5. Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
*{box-sizing:border-box;}
body{margin:0;font-family:Arial,sans-serif;background:#f5f7fb;color:#111827;}
header{padding:20px;background:#2563eb;color:white;text-align:center;}
nav{display:flex;justify-content:center;gap:20px;padding:15px;background:white;}
nav a{color:#111827;text-decoration:none;}
```

```
main{display:flex;gap:20px;padding:20px;}
.left{flex:2;background:white;padding:20px;border:1px solid #ddd;}
.right{flex:3;background:white;padding:20px;border:1px solid #ddd;}
.card{margin-bottom:15px;padding:15px;border:1px solid #ddd;border-radius:10px;}
img{width:100%;max-width:300px;height:200px;object-fit:cover;border-radius:10px;}
footer{padding:15px;text-align:center;background:#111827;color:white;}
</style>
</head>
<body>
<header><h1>My Website</h1></header>
<nav><a href="#">Home</a><a href="#">About</a><a href="#">Contact</a></nav>
<main>
<section class="left"><h2>Profile</h2><img src="image.jpg" alt="Profile image"></section>
<section class="right"><h2>Services</h2>
<article class="card"><h3>Web Design</h3><p>Creating clean webpages.</p></article>
<article class="card"><h3>Development</h3><p>Building web applications.</p></article>
</section>
</main>
<footer><p>&copy; 2026 Yug Sardhara</p></footer>
</body>
</html>
```

6. Expected Output: A complete page with header, navigation, two-column Flexbox content, image, cards and footer.

7. Attributes Used:

Property / Attribute	Belongs to	Meaning
class	HTML	Reusable CSS selector hook.
href	<a>	Link destination.
src	<img>	Image path.
alt	<img>	Alternative text.
display:flex	CSS	Two-column layout.
flex	CSS child	Flexible proportional sizing.
padding	CSS	Inner spacing.
margin	CSS	Outer spacing.
border	CSS	Boundary.
border-radius	CSS	Rounded corners.

8. Attribute → Belongs to → Meaning: HTML attributes identify/link content; CSS properties control presentation.

9. Important Rules:

• Start with structure, then style.

• Use classes for reusable components.

• Use Flexbox for the left/right arrangement.

• Use box-sizing:border-box for easier sizing.

• Test at different widths.

10. Common Mistakes:

• Using margin for every layout problem.

• Using fixed widths that break on smaller screens.

• Thinking flex:2 and flex:3 are fixed pixels; they represent flexible proportions.

• Forgetting alt text.

11. Viva / Remember Points: Complete layouts are combinations of selectors, box model, display, Flexbox, typography, colors and spacing.

QUICK REVISION SUMMARY

Use this section before a practical, viva or examination.

Property / Attribute	Belongs to	Meaning
CSS	Styling + layout of HTML	CSS
Types	Inline / Internal / External	CSS application
Syntax	selector { property:value; }	CSS
Element	p { }	Selector
Class	.box { }	Selector
ID	#title { }	Selector
Universal	* { }	Selector
Grouping	h1, h2, p { }	Selector
Colors	name / HEX / RGB / HSL	Color values
Background	color / image / repeat / size / position	Background
Fonts	family / size / style / weight / line-height	Typography
font-size keywords	xx-small → x-small → small → medium → large → x-large → xx-large → xxx-large	font-size
Margin	Outside space	Box model
Padding	Inside space	Box model
Border	Width + style + color	Box model
Box model	Content → Padding → Border → Margin	Layout
Display	block / inline / inline-block / none / flex / grid	Layout
Float	Moves left/right; content can wrap	Layout
Clear	Stops wrapping around floats	Float layout
Position	static / relative / absolute / fixed / sticky	Positioning
Images	width / max-width / object-fit / radius	Images
Lists	list-style-type / position / image	Lists
Links	link / visited / hover / active	Links
Flexbox	Parent display:flex; direct children become items	Layout
justify-content	Main-axis distribution	Flexbox parent
align-items	Cross-axis alignment	Flexbox parent
flex-direction	row / row-reverse / column / column-reverse	Flexbox parent
flex	Flexible child sizing	Flexbox child

Most Important Differences

Property / Attribute	Belongs to	Meaning
Concept	Difference	Remember
Margin vs Padding	Margin = outside; padding =	Outside vs inside

	inside.	
Class vs ID	Class reusable; ID normally unique.	. vs #
Float vs Flexbox	Float supports wrapping; Flexbox is a modern layout system.	float vs display:flex
justify-content vs align-items	Main axis vs cross axis.	Axis matters
relative vs absolute	Relative keeps its layout space; absolute is removed from normal flow.	Space kept vs removed
width vs max-width	width sets preferred size; max-width caps it.	Size vs limit

🧠 CSS Memory Map

SELECTOR → PROPERTY → VALUE

CONTENT → PADDING → BORDER → MARGIN

FLEX: Parent → display:flex → direction → main axis → cross axis → gap → Child flex

Viva Self-Check

☐ What is CSS?

☐ Name the three types of CSS.

☐ Difference between class and ID selector?

☐ What does * select?

☐ Difference between margin and padding?

☐ Explain the four parts of the box model.

☐ What is display:flex?

☐ Difference between float and Flexbox?

☐ Explain static, relative, absolute, fixed and sticky.

☐ What does object-fit:cover do?

☐ Difference between justify-content and align-items?

☐ What do flex:2 and flex:3 mean?